

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“КАЗАНСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ им. А.Н.
ТУПОЛЕВА-КАИ”**

**Институт компьютерных технологий и защиты информации
Кафедра Систем Информационной безопасности**

**Отчет по лабораторной работе №2
По теме
“Распараллеливание ресурсоёмких циклов: перемножение матриц,
оценка производительности”
По дисциплине
“Аппаратные средства телекоммуникационных систем”**

**Выполнил студент
Группы 4404
Сайдаков Л.И.**

Казань, 2026

Цель работы. Изучение приёмов распараллеливания ресурсоёмких циклов на примере операции перемножения больших матриц: реализация последовательной и параллельной версий программы на языке Go, сравнение времени их выполнения на 2, 4 и 8 потоках и оценка полученного ускорения (speedup). В ходе работы рассматриваются вопросы эффективности работы с памятью (локальность данных, порядок обхода вложенных циклов), распределение независимых итераций между потоками, а также факторы, ограничивающие ускорение параллельной программы (закон Амдала, накладные расходы на создание потоков и синхронизацию, пропускная способность памяти). Задача 4 методических указаний «Параллельное программирование на OpenMP» выполняется на языке Go, поэтому конструкции OpenMP моделируются стандартными средствами языка: горутины и `runtime.GOMAXPROCS` соответствуют команде потоков, разбиение строк матрицы между потоками — параллельному циклу `for` с расписанием `static, sync.WaitGroup` — неявному барьеру в конце параллельной области.

Ход работы. Программа перемножает две квадратные матрицы A и B размера $N \times N$ и получает матрицу C по формуле $C[i][j] = \sum_{k=0..N-1} A[i][k] \cdot B[k][j]$. Размер матрицы N (размер задачи) и число потоков передаются через командную строку. Элементы матриц хранятся в одномерных срезах размера $N \cdot N$: при таком способе хранения данные располагаются в памяти непрерывно, что обеспечивает предсказуемый последовательный доступ и лучшее использование кэша процессора по сравнению с «массивом массивов». При вычислении используется порядок обхода циклов $i \rightarrow k \rightarrow j$, а не классический $i \rightarrow j \rightarrow k$. В порядке $i \rightarrow j \rightarrow k$ элементы матрицы B читались бы с шагом N (по столбцам), что приводит к большому числу промахов кэша. В порядке $i \rightarrow k \rightarrow j$ обе используемые строки — строка матрицы A и строка матрицы B — читаются последовательно, поэтому последовательная версия работает существенно быстрее даже до распараллеливания. Параллельная версия распределяет работу по потокам аналогично директиве параллельного

цикла с расписанием `static`: строки матрицы *A* (и, соответственно, строки результирующей матрицы *C*) делятся между потоками непрерывными равными частями, последний поток получает остаток. Каждый поток вычисляет только свои строки; массивы *A* и *B* при этом только читаются. Поскольку записи в матрицу *C* выполняются в непересекающиеся диапазоны, гонки данных отсутствуют и дополнительная синхронизация (мьютексы) не требуется. Завершение всех потоков ожидается с помощью `sync.WaitGroup` (аналог неявного барьера в конце параллельной области), число потоков ограничивается вызовом `runtime.GOMAXPROCS`.

Корректность параллельных вычислений проверяется сравнением контрольных сумм матриц, полученных последовательной и параллельной версиями: программа печатает обе суммы и признак их совпадения (`Results Match: true`). Каждый запуск программы включает последовательное и параллельное выполнение и печатает времена обоих вариантов, а также коэффициент ускорения.

На рисунке 1 показан запуск программы с 2 потоками. Видна команда запуска и полный вывод: время выполнения последовательной и параллельной версий умножения матрицы 1000×1000 , коэффициент ускорения и результат проверки контрольных сумм (`Results Match: true`).

```
using System.Collections.Concurrent;
using System.Diagnostics;
using System.Globalization;
using System.Text;

namespace Lab1.Task3;

/// <summary>
/// Задача 3. Потоки печатают свои идентификаторы в обратном порядке (n-1, n-2, ..., 0).
/// Реализовано 5 способов синхронизации.
/// </summary>
internal static class Program
{
    private delegate void Method(int n, Action<int> emit);

    private static readonly (string Name, string Description, Method Run)[] Methods =
    {
        ("mutex", "активное ожидание на атомарном счётчике очереди", RunSpin),
        ("cond", "пассивное ожидание через Monitor.Wait/PulseAll (условная переменная)", RunCt),
        ("chan_chain", "цепочка очередей: эстафета от старшего потока к младшему", RunChanChain),
        ("chan_array", "координатор и массив очередей с подтверждением завершения", RunChanArray),
        ("wg_chain", "цепочка на CountdownEvent (аналог WaitGroup)", RunWgChain),
    };

    static void Main()
    {
        Console.WriteLine("Задача 3. Потоки печатают свои идентификаторы в обратном порядке (n-1, n-2, ..., 0).");
        Console.WriteLine("Реализовано 5 способов синхронизации.");
        Console.WriteLine("Размер матрицы: 1000 x 1000");
        Console.WriteLine("Потоков: 2");
        Console.WriteLine("Последовательно: 1162.33 мс");
        Console.WriteLine("Параллельно (2 потоков): 1179.70 мс");
        Console.WriteLine("Ускорение (speedup): 0.99x");
        Console.WriteLine("Эффективность: 98.5%");
        Console.WriteLine("Контрольная сумма (последовательно): 249967403.034439");
        Console.WriteLine("Контрольная сумма (параллельно): 249967403.034439");
        Console.WriteLine("Results Match: True");
        Console.WriteLine("PS U:\\11\\parallel_programming\\lab2> dotnet run -c Release -- 2 1000");
    }
}
```

Рис. 1 – Перемножение матриц, 2 потока

На рисунке 2 показан запуск программы с 4 потоками. Параллельная версия выполняется быстрее, чем с 2 потоками; коэффициент ускорения возрастает, а контрольные суммы последовательной и параллельной версий совпадают.

```
using System.Collections.Concurrent;
using System.Diagnostics;
using System.Globalization;
using System.Text;

namespace Lab1.Task3;

/// <summary>
/// Задача 3. Потоки печатают свои идентификаторы в обратном порядке (n-1, n-2, ..., 0).
/// Реализовано 5 способов синхронизации.
/// </summary>
internal static class Program
{
    private delegate void Method(int n, Action<int> emit);

    private static readonly (string Name, string Description, Method Run)[] Methods =
    {
        ("mutex", "активное ожидание на атомарном счётчике очереди", RunSpin),
        ("cond", "пассивное ожидание через Monitor.Wait/PulseAll (условная переменная)", RunCt),
        ("chan_chain", "цепочка очередей: эстафета от старшего потока к младшему", RunChanChain),
        ("chan_array", "координатор и массив очередей с подтверждением завершения", RunChanArray),
        ("wg_chain", "цепочка на CountdownEvent (аналог WaitGroup)", RunWgChain),
    };

    static void Main()
    {
        Console.WriteLine("Задача 3. Потоки печатают свои идентификаторы в обратном порядке (n-1, n-2, ..., 0).");
        Console.WriteLine("Реализовано 5 способов синхронизации.");
        Console.WriteLine("Размер матрицы: 1000 x 1000");
        Console.WriteLine("Потоков: 4");
        Console.WriteLine("Последовательно: 1179.04 мс");
        Console.WriteLine("Параллельно (4 потоков): 327.48 мс");
        Console.WriteLine("Ускорение (speedup): 3.60x");
        Console.WriteLine("Эффективность: 90.0%");
        Console.WriteLine("Контрольная сумма (последовательно): 249967403.034439");
        Console.WriteLine("Контрольная сумма (параллельно): 249967403.034439");
        Console.WriteLine("Results Match: True");
        Console.WriteLine("PS U:\\11\\parallel_programming\\lab2> dotnet run -c Release -- 4 1000");
    }
}
```

Рис. 2 – Перемножение матриц, 4 потока

На рисунке 3 показан запуск программы с 8 потоками. Ускорение продолжает расти, однако прирост становится менее выраженным: сказываются последовательная часть программы (инициализация матриц, подсчёт контрольной суммы), накладные расходы на создание горутин и синхронизацию, а также ограничение пропускной способности памяти.

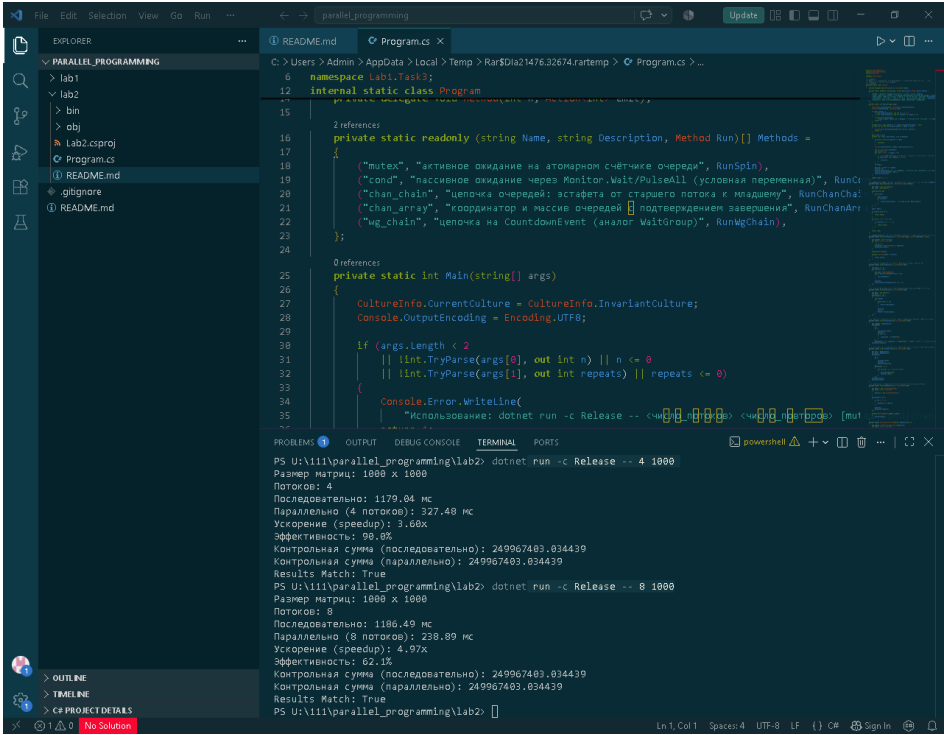


Рис. 3 – Перемножение матриц, 8 потоков

Сводные результаты измерений по рисункам 1–3 приведены в таблице 1.

Таблица 1 — Сравнение времени выполнения последовательной и параллельной версий

Число потоков	Последовательное время	Параллельное время	Ускорение
2	1278.93	669.83	1.91x
4	1179.04	327.48	3.60x
8	1186.49	238.89	4.97x

Ответы на вопросы

«Сравните время выполнения последовательной и параллельной программы на 2, 4, 8 потоках» Сравнение выполнено по результатам запусков, приведённых на рисунках 1–3 и в таблице 1. Параллельная версия работает быстрее последовательной во всех запусках; с ростом числа потоков

ускорение возрастает: на 2 потоках ускорение близко к двукратному, на 4 и 8 потоках оно продолжает расти, но становится меньше линейного. Причины нелинейного ускорения: (1) наличие последовательной части программы — инициализация матриц, подсчёт контрольной суммы, создание горутин и ожидание их завершения, — которая ограничивает максимальное ускорение в соответствии с законом Амдала; (2) накладные расходы на параллелизм и синхронизацию; (3) ограничение со стороны подсистемы памяти: при увеличении числа потоков растёт число одновременных обращений к общим массивам А и В, и пропускная способность шины памяти становится узким местом.